



Hospital Appointment System – Detailed Technical Q&A



Member 1 – Introduction & Overview

Technical / Java & Project Questions

1. **Q:** Which Java version did you use and why?

A: Java 17, because it is LTS, stable, and fully compatible with Spring Boot 3.x.

2. **Q:** How is object-oriented programming applied in your project?

A: Through classes for User, Patient, Doctor, Admin; inheritance is used for role specialization; encapsulation for private fields and accessors; polymorphism for method overrides like `getDashboardData()`.

3. **Q:** What design patterns are used?

A: MVC (Model-View-Controller) pattern separates concerns. Singleton pattern may be used for service beans (Spring handles it by default).

4. **Q:** How does Java interact with the database?

A: Via Spring Data JPA using Hibernate ORM, mapping Java objects to database tables (ORM), managing SQL behind the scenes.

5. **Q:** How does your system handle user sessions?

A: Using JWT tokens stored in client-side localStorage/cookies. Spring Security validates the token on each request.

Implementation Questions

6. **Q:** Why choose a web-based system over desktop or mobile?

A: Accessible via browser, cross-platform, easier deployment, centralized management.

7. **Q:** What is the role of HTML, CSS, and JS here?

A: HTML for structure, CSS for styling, JS for dynamic behavior and frontend interactivity.



Member 2 – Problem Statement & Objectives

Java & Spring Boot Questions

1. **Q:** What is Spring Boot and why is it used?

A: A Java framework for building production-ready applications quickly with embedded servers, auto-configuration, and dependency injection.

2. **Q:** What is dependency injection? How is it applied here?

A: A design pattern where objects are injected rather than instantiated manually. Used via `@Autowired` to inject repositories or services.

3. **Q:** Explain Spring Security authentication.

A: Configures roles and endpoints, uses JWT tokens, password encoded via BCrypt, validates tokens per request.

4. **Q:** What is role-based access control?

A: Users (Patient, Doctor, Admin) can only access resources permitted to their role via `@PreAuthorize` or security config.

Database & SQL Questions

5. **Q:** How is SQL handled in Spring Boot?

A: Using JPA repositories. CRUD operations are mapped to methods like `save()`, `findById()`. Custom queries via `@Query`.

6. **Q:** How is MySQL connected to Spring Boot?

A: Through `application.properties` or `application.yml` defining `spring.datasource.url`, `username`, `password`, and JPA settings.

Implementation Questions

7. **Q:** Why JWT instead of session-based auth?

A: Stateless, scalable, no server memory overhead, easy to integrate with REST APIs.

8. **Q:** How did you implement real-time slot checking?

A: Service layer checks availability before booking, uses transactional methods to prevent race conditions.



Member 3 – System Design & Implementation

Java & OOP Questions

1. **Q:** What is inheritance, and how is it used in your project?

A: Patient, Doctor, Admin inherit from User base class; common attributes (id, name, email) are in User.

2. **Q:** What is polymorphism in your project?

A: Overriding methods like `getDashboardData()` for different roles. Interfaces could also define common behavior.

3. **Q:** What is encapsulation and how is it applied?

A: Private fields with getters/setters. Example: `private String password;` only accessible via `getPassword()` or `setPassword()`.

Spring Boot & Implementation Questions

4. **Q:** Explain layered architecture. Why four layers?

A: Presentation (controllers, templates) → Service (business logic) → Repository (DB operations) → Database (data storage). Separates concerns, easy to test and maintain.

5. **Q:** What is `@Transactional` and why is it important?

A: Ensures atomic operations; booking either succeeds fully or rolls back on error to prevent double-booking.

6. **Q:** How are entities mapped to tables?

A: Using `@Entity`, `@Table`, `@OneToMany`, `@ManyToOne`, `@Inheritance(strategy = SINGLE_TABLE)` annotations.

7. **Q:** How is slot booking handled under high concurrency?

A: Spring's `@Transactional` and database-level constraints prevent overlapping bookings. Could use `Optimistic Locking` if needed.

SQL & DB Questions

8. **Q:** What is the schema design for appointments?

A: `Appointment` table has foreign keys to `Patient` and `Doctor`. `Slot` table linked via `appointment_id` or availability flag.

9. **Q:** How are relationships handled in SQL?

A: One-to-many for Doctor → Appointments, Many-to-one for Appointment → Doctor, etc.



Member 4 – Demo, Results & Future Scope

Java, Spring Boot & DB Questions

1. **Q:** How are dashboards implemented?

A: Controller fetches data from Service → Repository queries DB → Thymeleaf templates render data.

2. **Q:** How are prescriptions stored?

A: As strings in database or files, linked to appointment entity.

3. **Q:** How was testing done?

A: Manual testing of forms, API testing via Postman, database verification of CRUD operations.

Future Scope / Advanced Questions

4. **Q:** How to integrate email/SMS notifications?

A: Using JavaMailSender for email, Twilio API for SMS, triggered via service methods after booking.

5. **Q:** How to add video consultation?

A: Integrate WebRTC or third-party API, link consultation with appointment time.

6. **Q:** Why containerization?

A: Docker ensures environment consistency, easy deployment, scalable in CI/CD pipelines.



General Questions for All Members (Java / Spring Boot / SQL / OOP Focus)

1. **Q:** How does Spring Boot simplify Java development?

A: Auto-configuration, embedded server, dependency injection, reduces boilerplate setup.

2. **Q:** Explain ORM in your project.

A: Hibernate maps Java objects to SQL tables, converts CRUD operations into SQL automatically.

3. **Q:** What is the difference between `@Controller` and `@RestController`?

A: `@Controller` returns views/templates (Thymeleaf), `@RestController` returns JSON for APIs.

4. **Q:** How are transactions handled in Spring Boot?

A: `@Transactional` ensures atomicity; rollback on exceptions.

5. **Q:** Explain relationship mapping (`@OneToMany` , `@ManyToOne`) with example.

A: Doctor → Appointments is OneToMany; Appointment → Doctor is ManyToOne. Foreign key in appointment table.

6. **Q:** How is connection pooling managed?

A: Spring Boot uses HikariCP by default, efficient connection reuse.

7. **Q:** What is lazy vs eager fetching?

A: Lazy fetches related entities on demand; eager fetch loads immediately. Used in JPA relationships.

8. **Q:** How do you prevent SQL injection?

A: Using parameterized queries in JPA (`@Query` with parameters), avoiding raw concatenated SQL.

9. **Q:** Explain exception handling in Spring Boot.

A: Global exception handler using `@ControllerAdvice` or `@ExceptionHandler` per controller.

10. **Q:** How does OOP improve project design?

A: Makes code modular, reusable, maintainable; allows polymorphism and inheritance for role-specific behavior.

11. **Q:** What is the difference between `save()` and `saveAndFlush()` in JPA?

A: `save()` persists but may not commit immediately; `saveAndFlush()` persists and flushes to DB instantly.

12. **Q:** Why use MySQL over other DBs?

A: Relational, supports transactions, widely used, integrates well with Spring Boot.

13. **Q:** How are APIs secured?

A: Via Spring Security, JWT validation, role-based endpoint access.

This covers **deep technical, OOP, Spring Boot, SQL, and Java questions** that a teacher is likely to ask.

I can also make a **version with 100+ extremely tricky “viva-level” questions** for each part including **annotations, concurrency, JPA behavior, database transactions, indexing, optimization, and Java nuances** for full prep.

Do you want me to create that extended “teacher-level deep-dive” version next?